B. Mohan Vamsi Yadav
Sec-02/0866

Assignment-2

1) Reverse a string using stack

Algorithm

1. Initialize an empty stack.
2. Push each charecter of the string into the stack.
3. Pop charecters one by one and store back into the strings.
4. Print the reversed string

Code

```
#include <stdio.h>
# include <string.h>
# define MAX 100

char stack[MAX];
int top = -1;

Void push(char ch) {
        Stack[++top] = ch;
}

char pop() {
        return stack[top--];
}
```

```
int main() {
  char str[100];
  int i;
  Printf("Enter a String: ");
  Scanf("%s", str);
  for (i=0; i< strlen(str); i++) {
      Push(str[i]);
  }

  for (i=0; i< strlen(str); i++) {
      Str[i] = pop();
  }
  printf("Reversed string: %s", str);
  return 0;
}
```

Output:

input: DATA
output: ATAD

2) Balenced Parentheses

Algorithm

1. Scan the expression from left to righ.
2. push opening brackets (onto stack.
3. when closing bracket) appears, pop from s
4. If stack is empty at the end → Balenced.

Code

```
#include <stdio.h>
#include <string.h>
# define MAX 100

char stack [MAX];
int top = -1;

void push ( char ch) {
        stack [++ top] = ch;
}

void pop() {
        if (top != -1)
            top--;
}

int main() {
 char exp[100];
 int i;
 printf("Enter expression: ");
 scanf("%s", exp);
 for (i=0; i<strlen(exp); i++) {
  if (exp[i] == '(')
        push(exp[i]);
    else if (exp[i] == ')').
        pop;
}
```

```
if(top==-1)
    Printf("Balenced Expression");
else
    Printf("Not Balanced");
return 0;
}
```

Output

input: (A+B)* (C-D)
Output: Balanced Expression

3) Next Greater element

* Algorithm

1. Traverse array.
2. For each element, check next greater element.
3. Use stack to store elements.
4. Print result.

Code

```
# include <stdio.h>
int main() {
    int arr[] = {4,5,2,10,8};
    int n=5;
  for(int i=0; i<n; i++) {
    int found = 0;
    for(int j= i+1; j<n; j++) {
```

```
        if(arr[j] > arr[i]) {
            Printf("%d -> %d\n", arr[i], arr[j]);
            found = 1;
            break;
        }
    }

    if(!found)
        printf("%d -> -1\n", arr[i]);
    }
    return 0;
}
```

output

```
4->5
5->10
2->-1
8->-1
```

4) Printer Queue Simulation

Algorithm

1. Create a queue
2. Add document -> enqueue
3. Print document -> Dequeue
4. Display queue

Code

```
#include <stdio.h>
# define MAX 5

int queue[MAX], front=-1, rear=-1;
Void enqueue(int x) {
    if(rear == MAX-1)
        Printf("Queue Full\n");
    else {
        if (front==-1) Front=0;
        queue [++rear]=x;
    }
}

Void dequeue() {
    if (front==-1)
        Printf("queue Empty\n");
    else {
        Printf ("printed Document: %d\n", queue[front]);
        front++;
    }
}

Void display() {
    if(front==-1)
        Printf("No documents\n");
    else {
        for (int i=front; i<=rear; i++)
```

```
    Printf("%d ", queue[i]);
  Printf("\n");
 }
}

int main() {
    enqueue(1);
    enqueue(2);
    display();
    dequeue();
    display();
    return 0;
}
```

5) Circular Queue (Menu Driven)

Code

```
# include <stdio.h>
# define MAX 5
 int Cq[MAX];
 int front = -1, rear=-1;
 Void enqueue (int x) {
    if((rear+1) % MAX == front)
      Printf("queue Full\n");
    else {
```

```
    if (front == -1) front = 0;
    rear = (rear+1) % MAX;
    cq[rear] = x;
  }
}

void dequeue() {
    if (front == -1)
        Printf("queue Empty\n");
    else {
        Printf("Deleated: %d\n", cq[front]);
        if (front == rear)
            front = rear = -1;
        else
            front = (front+1) % MAX;
    }
}

void peak() {
    if (front != -1)
        Printf("queue Empty\n");
    else {
        int i = front;
        while(1) {
            Printf("%d", cq[i]);
            if (i == rear) break;
```

```
        i = (i+1) % MAX ;
    }
    printf("\n");
  }
}

int main() {
    int choice, x;
    while(1) {
        Printf("\n1. Enqueue 2. Dequeue 3.Peak 4.Display
         5. Exit\n");
        SCanf("%d",&choice);
      Switch (choice) {
        Case 1:
            Printf("Enter value: ");
            Scanf("%d", &x);
            enqueue(x);
            break;
        Case 2:
            dequeue();
            break;
        Case 3:
            Peek();
            break;
```

```
Case 4:
        display();
        break;
Case 5:
        return 0;
    }
  }
}
```